

Project Overview :

This project is an interactive football card game built using MATLAB App Designer. The game allows a human player to draft a team of football players and pit them against computer. I have created a dataset of 50 players. The player interacts with the GUI and drafts his/her deck from randomly generated footballer cards from this dataset. Computer also drafts its deck from randomly generated footballer cards from this dataset. The higher OVR(Overall Rating) among the two cards pitted against each other contributes to corresponding player/computer point.

Core Features :

- i) It automatically reads and takes data from a .csv file containing 50+ players, their roles (Attack/Defense), and Overall (OVR) ratings.
- ii) It filters the deck by player role and gives specific positions during designated rounds in a random manner.
- iii) It features a clickable "Bench" that stores temporary state data, allowing the player to transfer cards from the bench to specific field slots.
- iv) It prevents illegal moves (such as : placing a Defender in an Attack slot) by background data validation and notifies through UI alert messages.
- v) It uses UI property toggling (Visible, Enable) to lock and unlock buttons, forcing the user into a strict chronological game sequence.
- iv) It takes text data from UI components, compares stats across 5 specific "lanes," and tallies a final score to determine a winner.

Architecture :

i) Properties (State Management)

The app relies on custom private properties to maintain a "short-term memory" across different functions:

- DeckData: Stores the raw table imported from the CSV.
- AttackerDeck & DefenderDeck: Two split tables containing filtered subsets of the main deck.

- SelectedPlayer: Holds the object data of the currently clicked bench button, enabling the transfer of text to the main grid.

ii) User Interface(UI) Layout

The pitch is represented by a 5x5 colored grid of Buttons:

- Row 1 (Red - OD1 to OD5): Opponent Defense
- Row 2 (Red - OA1 to OA5): Opponent Attack
- Row 3 (Blue - PA1 to PA5): Player Attack
- Row 4 (Blue - PD1 to PD5): Player Defense
- Row 5 (Green - Bench 1 to Bench 5): The active drafting hand.
- Control Panel (Right Side): Next Round, Complete Deck, Play Match buttons.

4. Game Flow and Logic

Phase 1: Initialization and Attacker Draft

Triggered by the startupFcn.

- The app loads the CSV using readtable.
- It splits the deck and shuffles the AttackerDeck using randperm.
- It deals 5 Attackers to the Green Bench and formats their text into 3 lines (Name, Role, OVR).
- The sequential control buttons are disabled (Enable = 'off') to prevent sequence breaking.

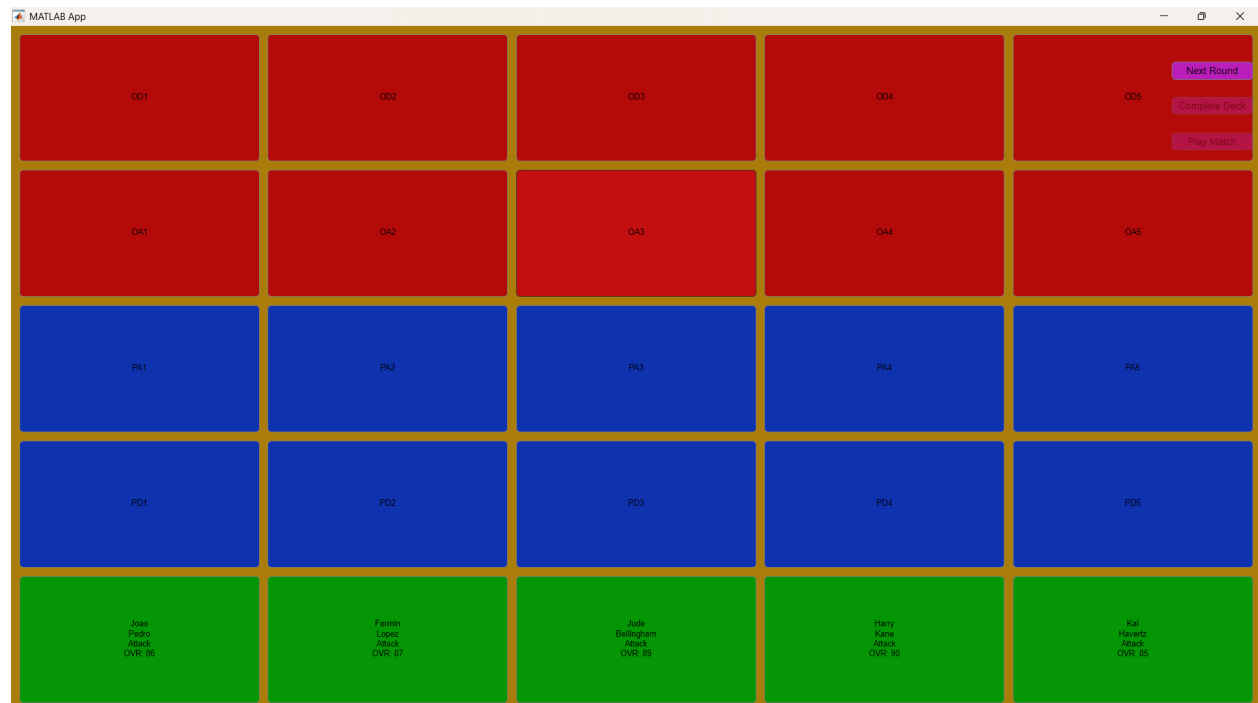


Startup Function :

```
% Callbacks that handle component events
methods (Access = private)

% Code that executes after component creation
function startupFcn(app)

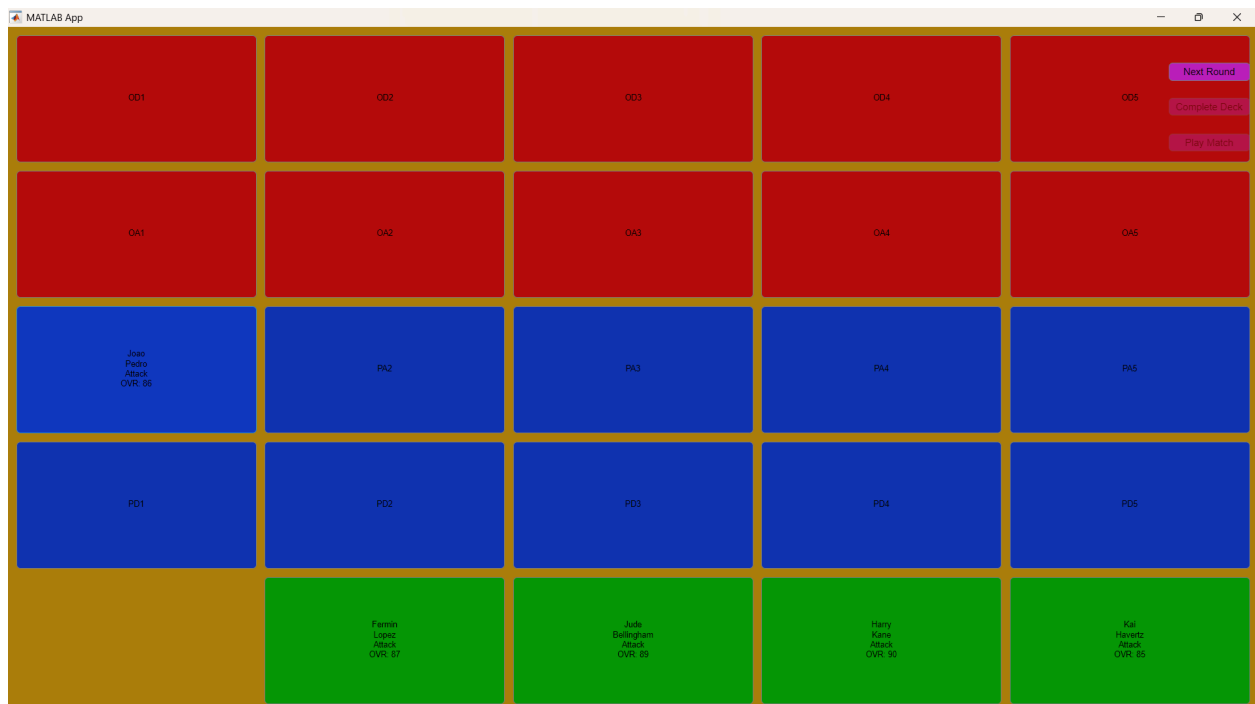
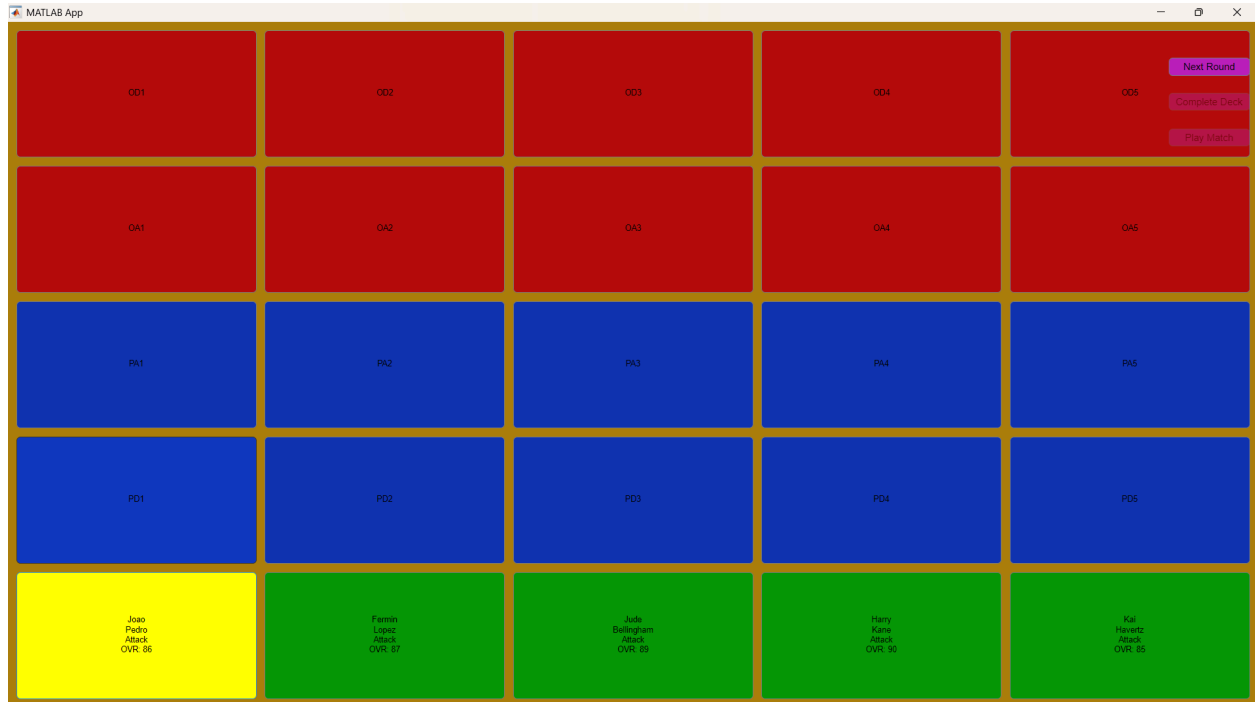
    app.DeckData = readtable('Card Game Players.csv');
    app.AttackerDeck = app.DeckData(app.DeckData.Section == "Attack", :);
    app.DefenderDeck = app.DeckData(app.DeckData.Section == "Defense", :);
    totalAttackers = height(app.AttackerDeck);
    randomRows = randperm(totalAttackers);
    p1 = app.AttackerDeck(randomRows(1), :);
    p2 = app.AttackerDeck(randomRows(2), :);
    p3 = app.AttackerDeck(randomRows(3), :);
    p4 = app.AttackerDeck(randomRows(4), :);
    p5 = app.AttackerDeck(randomRows(5), :);
    app.Bench_1Button.Text = [split(string(p1.Name), ' '); string(p1.Section); "OVR: " + string(p1.OVR)];
    app.Bench_2Button.Text = [split(string(p2.Name), ' '); string(p2.Section); "OVR: " + string(p2.OVR)];
    app.Bench_3Button.Text = [split(string(p3.Name), ' '); string(p3.Section); "OVR: " + string(p3.OVR)];
    app.Bench_4Button.Text = [split(string(p4.Name), ' '); string(p4.Section); "OVR: " + string(p4.OVR)];
    app.Bench_5Button.Text = [split(string(p5.Name), ' '); string(p5.Section); "OVR: " + string(p5.OVR)];
    app.CompleteDeckButton.Enable = 'off';
    app.PlayMatchButton.Enable = 'off';
end
```

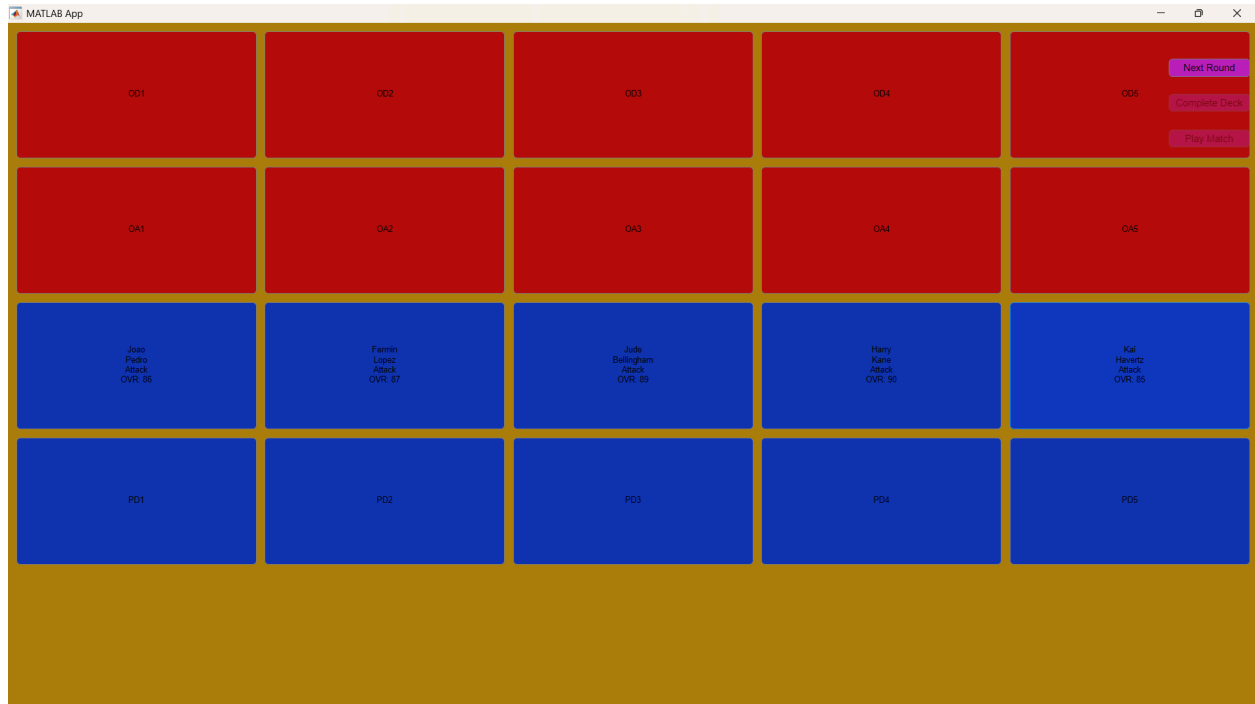


Phase 2: The Player Attack Draft

Triggered by clicking a Bench button and then a Blue PA slot.

- i) The app saves the Bench button to SelectedPlayer and turns it yellow.
- ii) Clicking a PA slot checks if the card's Role line equals "Attack".
- iii) If valid, the text transfers, the Bench button hides (Visible = 'off'), and the memory clears.





Phase 3: Opponent Attack & Defender Draft

Triggered by the “Next Round” button.

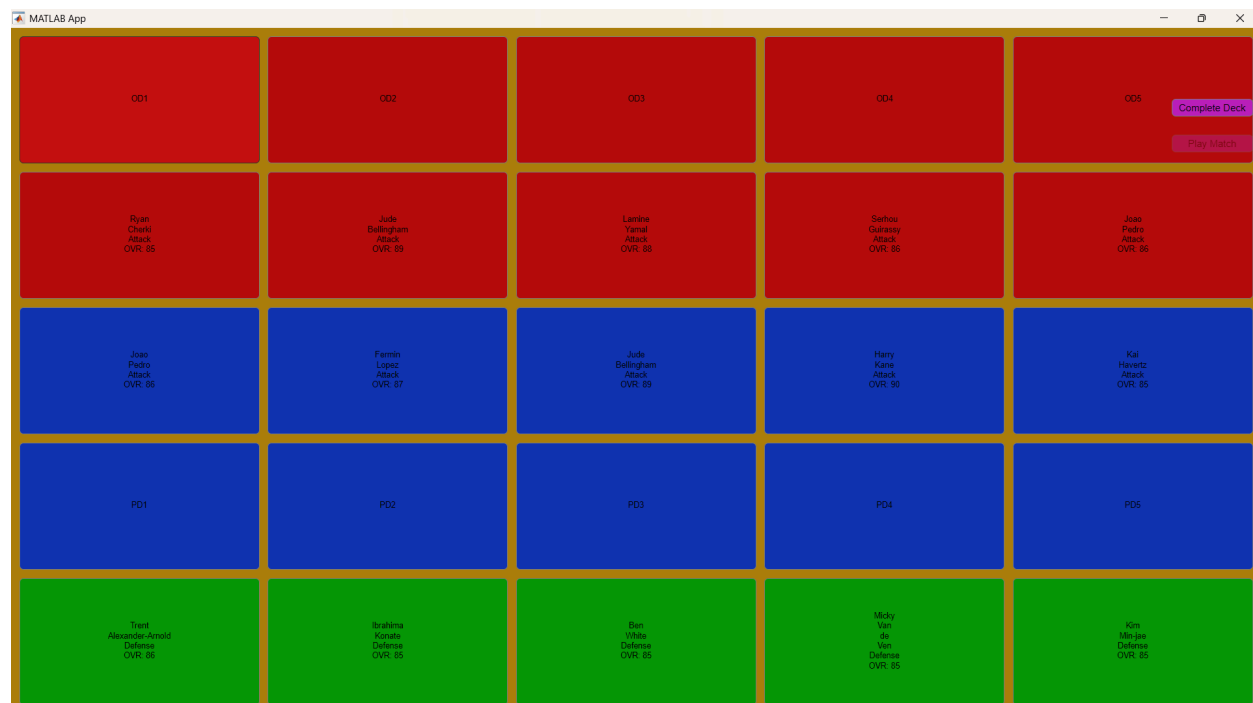
- i) The AI randomly pulls 5 Attackers and populates the OA row.
- ii) The player's bench is replenished with 5 Defenders from the DefenderDeck.
- iii) The bench visibility is reset (Visible = 'on'), and the button color resets to green.
- iv) The “Complete Deck” button is enabled.

Next Round button callback function :

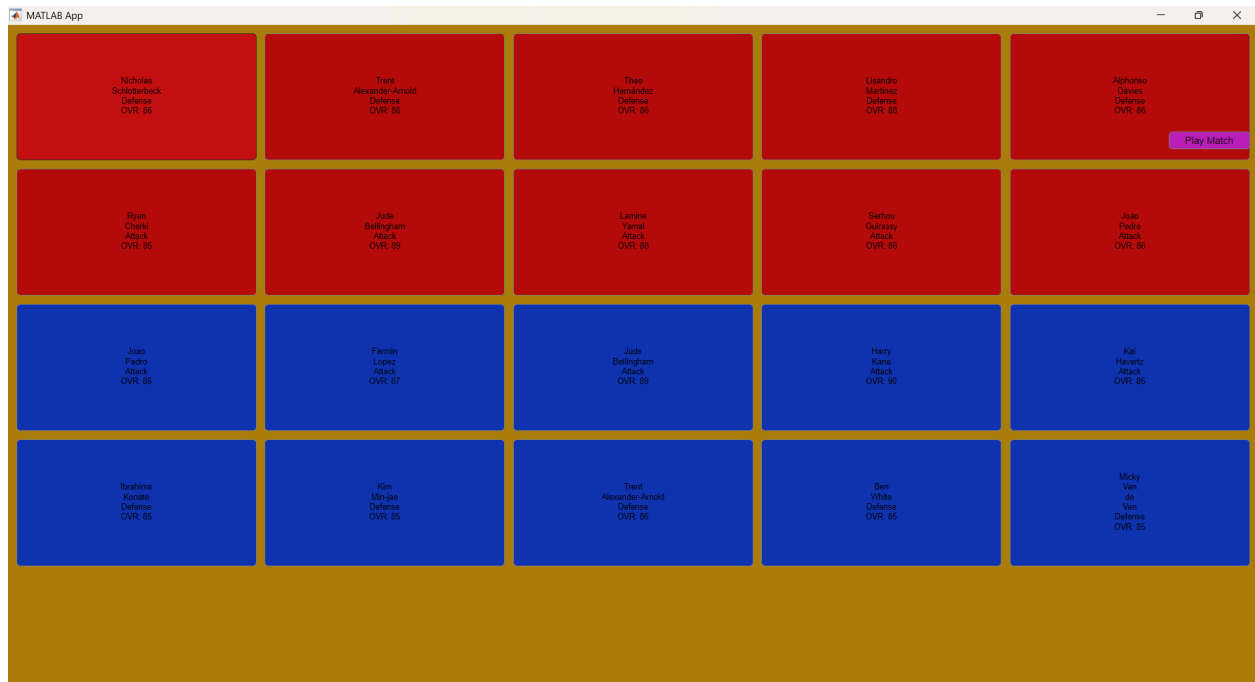
```
% Button pushed function: NextRoundButton
function NextRoundButtonPushed(app, event)

    compRows = randperm(height(app.AttackerDeck));
    cA1 = app.AttackerDeck(compRows(1), :);
    cA2 = app.AttackerDeck(compRows(2), :);
    cA3 = app.AttackerDeck(compRows(3), :);
    cA4 = app.AttackerDeck(compRows(4), :);
    cA5 = app.AttackerDeck(compRows(5), :);
    app.OA1Button.Text = [split(string(cA1.Name), ' '); string(cA1.Section); "OVR: " + string(cA1.OVR)];
    app.OA2Button.Text = [split(string(cA2.Name), ' '); string(cA2.Section); "OVR: " + string(cA2.OVR)];
    app.OA3Button.Text = [split(string(cA3.Name), ' '); string(cA3.Section); "OVR: " + string(cA3.OVR)];
    app.OA4Button.Text = [split(string(cA4.Name), ' '); string(cA4.Section); "OVR: " + string(cA4.OVR)];
    app.OA5Button.Text = [split(string(cA5.Name), ' '); string(cA5.Section); "OVR: " + string(cA5.OVR)];
    defRows = randperm(height(app.DefenderDeck));
    d1 = app.DefenderDeck(defRows(1), :);
    d2 = app.DefenderDeck(defRows(2), :);
    d3 = app.DefenderDeck(defRows(3), :);
    d4 = app.DefenderDeck(defRows(4), :);
    d5 = app.DefenderDeck(defRows(5), :);
    app.Bench_1Button.Text = [split(string(d1.Name), ' '); string(d1.Section); "OVR: " + string(d1.OVR)];
    app.Bench_2Button.Text = [split(string(d2.Name), ' '); string(d2.Section); "OVR: " + string(d2.OVR)];
    app.Bench_3Button.Text = [split(string(d3.Name), ' '); string(d3.Section); "OVR: " + string(d3.OVR)];
    app.Bench_4Button.Text = [split(string(d4.Name), ' '); string(d4.Section); "OVR: " + string(d4.OVR)];
    app.Bench_5Button.Text = [split(string(d5.Name), ' '); string(d5.Section); "OVR: " + string(d5.OVR)];
    app.Bench_1Button.Visible = 'on'; app.Bench_1Button.BackgroundColor = [0.03,0.60,0.03];
    app.Bench_2Button.Visible = 'on'; app.Bench_2Button.BackgroundColor = [0.03,0.60,0.03];
    app.Bench_3Button.Visible = 'on'; app.Bench_3Button.BackgroundColor = [0.03,0.60,0.03];
    app.Bench_4Button.Visible = 'on'; app.Bench_4Button.BackgroundColor = [0.03,0.60,0.03];
    app.Bench_5Button.Visible = 'on'; app.Bench_5Button.BackgroundColor = [0.03,0.60,0.03];
    app.NextRoundButton.Visible = 'off';
    app.CompleteDeckButton.Enable = 'on';
end
```

After pressing Next Round button :



After pressing Complete Deck button :



Phase 5: Simulation and Scoring

Triggered by the “Play Match” Button.

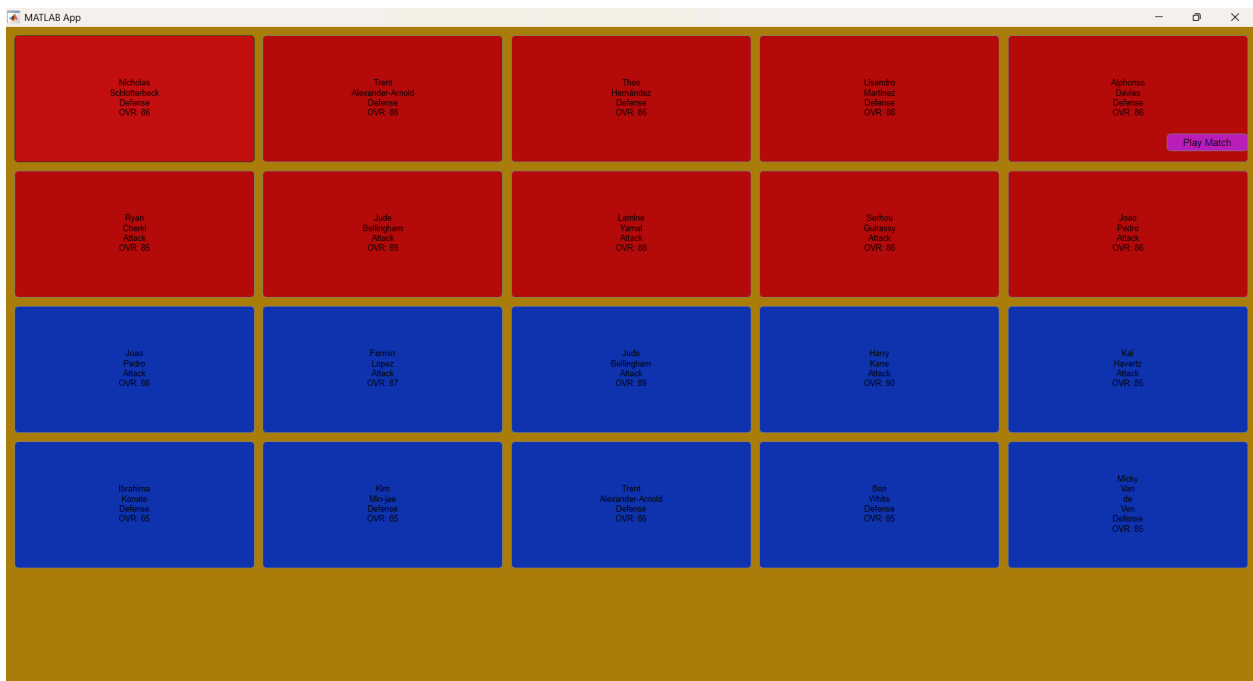
- A for loop iterates through columns 1 to 5.
- The app compares footballer cards OVR in PA vs. OD and OA vs. PD (PA1 vs. OD1, PA2 vs. OD2, PA3 vs. OD3, PA4 vs. OD4, PA5 vs. OD5, OA1 vs. PD1, OA2 vs. PD2, OA3 vs. PD3, OA4 vs. PD4, OA5 vs. PD5). Higher OVR leads to a goal in corresponding (player or computer) side.
- Goals are tallied, and the final win/loss/draw condition is outputted by a UI message.

Callback function of Play Match button :

```
% Button pushed function: PlayMatchButton
function PlayMatchButtonPushed(app, event)

    playerGoals = 0;
    computerGoals = 0;
    PA_text = {app.PA1Button.Text, app.PA2Button.Text, app.PA3Button.Text, app.PA4Button.Text, app.PA5Button.Text};
    OD_text = {app.OD1Button.Text, app.OD2Button.Text, app.OD3Button.Text, app.OD4Button.Text, app.OD5Button.Text};
    OA_text = {app.OA1Button.Text, app.OA2Button.Text, app.OA3Button.Text, app.OA4Button.Text, app.OA5Button.Text};
    PD_text = {app.PD1Button.Text, app.PD2Button.Text, app.PD3Button.Text, app.PD4Button.Text, app.PD5Button.Text};
    for i = 1:5
        p_att_ovr = str2double(extractAfter(PA_text{i}(end), "OVR: "));
        c_def_ovr = str2double(extractAfter(OD_text{i}(end), "OVR: "));
        if p_att_ovr > c_def_ovr
            playerGoals = playerGoals + 1;
        end
        c_att_ovr = str2double(extractAfter(OA_text{i}(end), "OVR: "));
        p_def_ovr = str2double(extractAfter(PD_text{i}(end), "OVR: "));
        if c_att_ovr > p_def_ovr
            computerGoals = computerGoals + 1;
        end
    end
    if playerGoals > computerGoals
        matchResult = "YOU WIN!";
    elseif computerGoals > playerGoals
        matchResult = "COMPUTER WINS!";
    else
        matchResult = "IT'S A DRAW!";
    end
    finalMessage = sprintf('Player: %d \nComputer: %d \n\n%s', playerGoals, computerGoals, matchResult);
    uialert(app.UIFigure, finalMessage, 'Full Time Result');
    app.PlayMatchButton.Visible = 'off';
end
```

Before pressing Play Match button :



Here :

PA1 vs OD1 = Joao Pedro(86) vs. Schlotterbeck(86)I = Blocked

PA2 vs OD2 =Fermin(87) vs. Alexander Arnold(86) = Player Goal +1

PA3 vs OD3 = Bellingham(89) vs. Theo Hernandez(86) = Player Goal +1

PA4 vs OD4 = Kane(90) vs. Lisandro(88) = Player Goal +1

PA5 vs OD5 = Havertz(85) vs. Davies(86) = Blocked

PD1 vs OA1 = Konate(85) vs. Cherki(85) = Blocked

PD2 vs OA2 =Kim Min Jae(85) vs. Bellingham(89) = Computer Goal +1

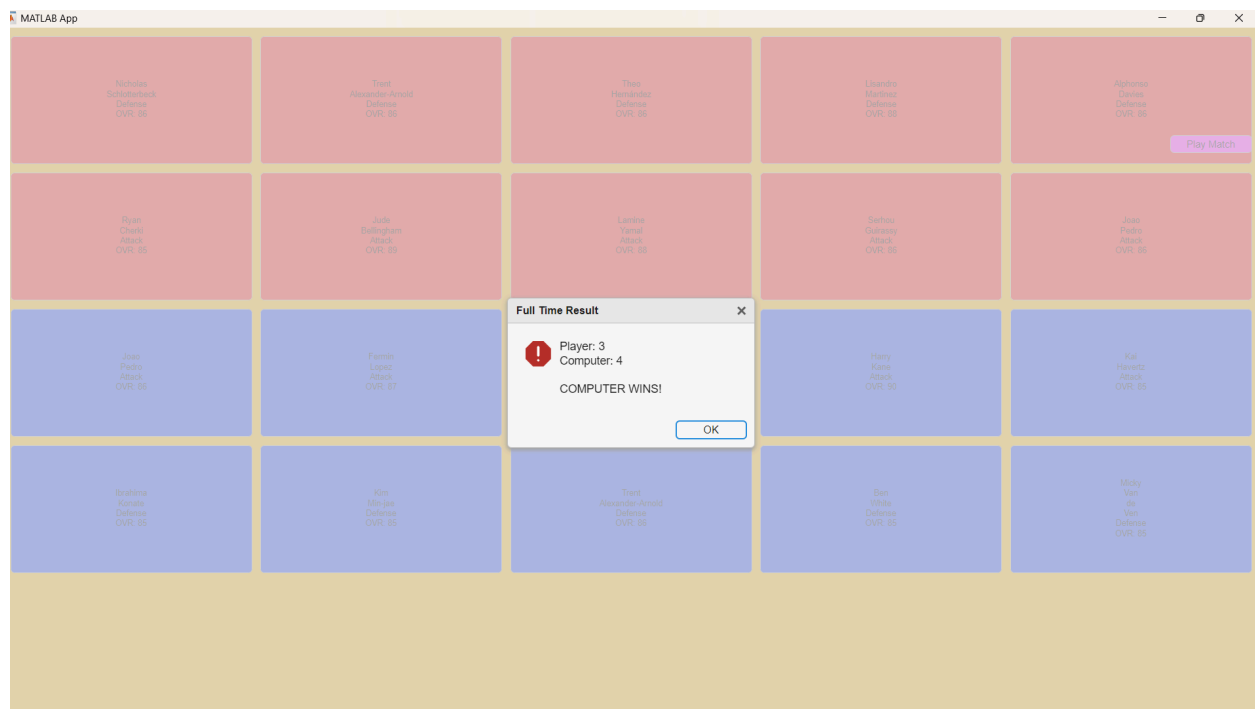
PD3 vs OA3 = Alexander Arnold(86) vs. Yamal(88) = Computer Goal +1

PD4 vs OA4 = Ben White(85) vs. Guirassy(86) = Computer Goal +1

PD5 vs OA5 = Van De Ven(85) vs. Joao Pedro(86) = Computer Goal +1

So, final score should be (Player) 3 - 4 (Computer). Computer would win.

After pressing the Play Match button , we would observe the same result by a UI message pop-up.



Future Development Opportunities

- **Image Integration:** I could not figure out how to map specific .jpg or .png files to the buttons to display player faces. If this can be done, the game would be much more interactive.
- **Advanced Stats:** I only used Attack vs Defense with one OVR. For further improvement, there is scope of expanding the clash mechanics to include specific match-ups (such as : Pace vs. Defending) rather than just a flat OVR comparison.
- **Multiplayer:** I have only included one human player vs. computer. For more interactivity, there is scope of enabling the opponent rows to be clickable by a second local player instead of using an automated computer.
- **Limitation of usage of each card :** To make the game more competitive, a feature could be introduced which would prohibit two teams to have the same card.